Le terminal et Bash, sans souris

Les modèles d'IA s'entraînent sur des serveurs distants, sans écran ni souris : on les pilote uniquement en tapant des commandes. Cette fiche t'apprend à te déplacer, créer, copier, chercher et automatiser depuis le terminal, puis à écrire ton premier script Bash.

1. C'est quoi, le terminal ?
2. Où suis-je ? pwd, ls
3. Les chemins
4. Se déplacer : cd
5. Créer, copier, déplacer, supprimer
6. Lire un fichier
7. Chercher : find et grep
8. Les tuyaux | et les redirections >
9. Ton premier script Bash
10. Mission : le chantier ExamPredict
11. Mini-test corrigé
12. À savoir par cœur

> **Pourquoi c'est vital dans le métier**
>
> Un ML Engineer loue des machines puissantes (des GPU) sur le cloud. Il s'y connecte à distance et ne voit qu'un terminal : pas de bureau, pas d'explorateur de fichiers. Il y lance un entraînement qui dure des heures, cherche une erreur dans des fichiers de logs de 100 000 lignes et déplace des datasets de plusieurs gigaoctets. Tout ça en tapant des commandes. **Sans le terminal, impossible de faire ce métier.**

1. C'est quoi, le terminal ?

Analogie : d'habitude, tu parles à l'ordinateur avec la **souris**, en montrant du doigt. Le terminal, c'est lui parler avec des **phrases écrites**. C'est plus rapide, plus précis, et ça peut s'automatiser.

> **Définition 1.1**
>
> Le **terminal** est une fenêtre où l'on tape des **commandes**. Le programme qui lit et exécute ces commandes s'appelle le **shell**. Sur Linux et sur les serveurs, ce shell s'appelle **Bash**. Sur Windows, **Git Bash** te donne exactement le même.

> **Réglage obligatoire dans VS Code**
>
> Le terminal de VS Code s'ouvre par défaut en **PowerShell**, qui n'utilise pas les mêmes commandes. Passe-le en Git Bash : ouvre le terminal (`Ctrl` + `ù`), clique sur la petite flèche **v** à côté du **+**, puis choisis **Git Bash**.

Une commande se lit toujours de la même façon :

```
commande   -options   cible
   ls        -l       data/        # « liste, en format long, le dossier data »
```

2. Où suis-je ? `pwd` , `ls`

Analogie : le terminal, c'est se promener dans une maison **les yeux bandés**. Avant de bouger, on demande « où suis-je ? » puis « qu'y a-t-il autour de moi ? ».

COMMANDE	VIENT DE	SE DIT	EFFET
`pwd`	*print working directory*	« où suis-je ? »	affiche le dossier actuel
`ls`	*list*	« qu'y a-t-il ici ? »	liste les fichiers et dossiers
`ls -l`	*long*	« avec les détails »	taille, date, droits
`ls -a`	*all*	« même les cachés »	montre aussi `.git` , `.gitignore` …
`clear`		« efface l'écran »	nettoie le terminal

3. Les chemins

> **Définition 3.1** *(chemin)*
>
> Un **chemin** est l'adresse d'un fichier ou d'un dossier. Les dossiers sont séparés par `/` .

SYMBOLE	VEUT DIRE	ANALOGIE
`~`	ton dossier personnel (`/c/Users/josep`)	ta maison
`.`	le dossier où je suis	« ici »
`..`	le dossier parent, celui du dessus	« l'étage du dessus »
`/`	la racine, tout en haut	le rez-de-chaussée de l'immeuble

> **Définition 3.2** *(absolu ou relatif)*
>
> Un chemin **absolu** part de la racine ou de `~` : il marche de n'importe où. Exemple : `~/AI-ENGINEER-2027/WEEK1` .
> Un chemin **relatif** part de **là où tu es**. Exemple : `WEEK1/DAY4` ne marche que si tu es dans `AI-ENGINEER-2027` .
>
> **En clair :** l'adresse postale complète (absolu) contre « la deuxième porte à gauche » (relatif). La deuxième, ça dépend d'où tu pars.

> **Le bug que tu as déjà rencontré**
>
> Au Day 2, `python main.py` lancé depuis `src/` a donné `FileNotFoundError: 'data\\exams'` . Le programme utilisait le chemin **relatif** `data/exams` , qui n'existe pas quand on part de `src/` . Depuis la racine du projet, ça marchait. Comprendre les chemins, c'est corriger ce bug en 5 secondes.

---

4. Se déplacer : `cd`

```
cd ~/AI-ENGINEER-2027        # aller dans un dossier (chemin absolu)
cd WEEK1                     # entrer dans un sous-dossier (chemin relatif)
cd ..                        # remonter d'un étage
cd ../..                     # remonter de deux étages
cd ~                         # rentrer à la maison
cd -                         # revenir au dossier précédent
```

Les deux super-pouvoirs

`Tab` **complète** automatiquement les noms. Tape `cd WE` puis `Tab`, et il écrit `WEEK1/` pour toi. Moins de fautes de frappe.
`↑` **rappelle** les commandes précédentes. Pas besoin de tout retaper.

5. Créer, copier, déplacer, supprimer

COMMANDE	VIENT DE	EXEMPLE	EFFET
`mkdir`	*make directory*	`mkdir -p data/raw`	crée un dossier (`-p` crée aussi les parents)
`touch`		`touch notes.txt`	crée un fichier vide
`cp`	*copy*	`cp a.txt b.txt`	copie un fichier
`cp -r`	*recursive*	`cp -r data data_backup`	copie un dossier entier
`mv`	*move*	`mv a.txt archive/`	déplace, **ou renomme** : `mv vieux.txt neuf.txt`
`rm`	*remove*	`rm a.txt`	supprime un fichier
`rm -r`		`rm -r brouillon/`	supprime un dossier et tout son contenu

DANGER : `rm` n'a pas de corbeille

Ce que `rm` supprime est **perdu pour toujours**, sans confirmation. Règles de pro : faire `ls` avant `rm` pour vérifier la cible, ne jamais taper `rm -rf` sur un chemin dont on n'est pas sûr, et toujours avoir son code sur Git.

6. Lire un fichier

COMMANDE	EFFET	USAGE ML
`cat f.txt`	affiche tout le fichier	petits fichiers de config
`head -n 5 f.txt`	les 5 premières lignes	voir les colonnes d'un CSV
`tail -n 5 f.txt`	les 5 dernières lignes	voir la fin d'un log d'entraînement
`wc -l f.txt`	compte les lignes (*word count*)	combien d'exemples dans le dataset ?
`echo "texte"`	affiche un texte	messages dans les scripts

7. Chercher : `find` et `grep`

> **Définition 7.1**
>
> `find` cherche des **fichiers** par leur nom. `grep` cherche un **mot à l'intérieur** des fichiers.
>
> **En clair :** `find` trouve le bon livre dans la bibliothèque. `grep` trouve la bonne phrase dans le livre.

```
find . -name "*.py"            # tous les fichiers Python, ici et en dessous
find . -name "*.pdf"           # tous les PDF                        # * = « n'im

grep "def" r4_fonction.py      # les lignes qui contiennent "def"
grep -n "return" *.py          # -n : avec le numéro de ligne
grep -r "count_keyword" src/   # -r : dans tout un dossier
grep -i "erreur" log.txt       # -i : ignore majuscules et minuscules
grep -c "forces" resultats.txt # -c : compte les lignes trouvées
```

> **Dans le métier**
>
> Un entraînement a planté cette nuit ? `grep -n "Error" train.log` trouve la ligne exacte au milieu de 100 000 lignes, en une seconde.

8. Les tuyaux `|` et les redirections `>`

> **Définition 8.1** *(pipe)*
>
> Le **tuyau** `|` (`AltGr` + `6`) envoie le résultat d'une commande **dans** la suivante.
>
> **En clair :** une chaîne d'usine. La première machine passe son travail à la deuxième, qui le passe à la troisième.

```
ls | wc -l                          # combien de fichiers ici ?
grep "forces" resultats.txt | wc -l   # combien de lignes "forces" ?
sort resultats.txt | uniq -c | sort -rn | head -n 5
# trier → compter les doublons → classer du plus grand au plus petit → top 5
# C'est exactement Counter(...).most_common(5) de la fiche F2.DAY1, en une ligne
```

SYMBOLE	EFFET	EXEMPLE
`>`	écrit le résultat dans un fichier, **en écrasant**	`ls > liste.txt`
`>>`	ajoute à la fin du fichier	`echo "fini" >> log.txt`

C'est la même logique que les modes `"w"` et `"a"` de `open()` en Python (fiche F2.DAY2).

9. Ton premier script Bash

> **Définition 9.1**
>
> Un **script Bash** est un fichier `.sh` qui contient une liste de commandes, exécutées de haut en bas. C'est une recette, comme un fichier `.py`, mais pour le terminal.

```bash
#!/bin/bash
# bonjour.sh : la 1re ligne dit « lis-moi avec Bash »

nom=$1                          # $1 = le 1er mot donné après le nom du script
echo "Bonjour $nom"
echo "Tu es dans : $(pwd)"      # $( ) = « remplace par le résultat de la commande
echo "Il y a $(ls | wc -l) éléments ici"
```

```bash
chmod +x bonjour.sh             # donne le droit d'exécuter (change mode)
./bonjour.sh Joseph             # ./ = « le script qui est ici »
# Bonjour Joseph
```

Les 2 pièges de Bash

1. **Pas d'espace autour du `=`** : `nom=$1` ✅, `nom = $1` ❌ (Bash croit que `nom` est une commande).
2. Pour **lire** une variable, on met un `$` devant : `echo $nom`.

Variables d'environnement et PATH

Certaines variables sont définies pour tout le système. `echo $HOME` affiche ton dossier personnel. `echo $PATH` affiche la liste des dossiers où le terminal cherche les programmes. Quand Windows répond « Python was not found », c'est que le bon dossier de Python n'est pas dans le `PATH`. C'est pour ça que `py` marchait et pas `python`.

10. Mission : le chantier ExamPredict

À faire 100 % au clavier, sans souris (35 min)

1. Va dans `~/AI-ENGINEER-2027/WEEK1/DAY4` et vérifie avec `pwd`.
2. Crée l'arborescence `bash_lab/data/raw` et `bash_lab/data/clean` en **une seule** commande.
3. Copie `../DAY1/exercices/resultats.txt` dans `bash_lab/data/raw/`.
4. Compte ses lignes avec `wc -l` (attendu : 10 000).
5. Affiche ses 5 premières lignes, puis ses 5 dernières.
6. Compte les lignes qui contiennent « forces » (`grep -c`), puis la même chose en ignorant les majuscules (`grep -ic`). Pourquoi deux résultats différents ?
7. Affiche le top 5 des thèmes avec la chaîne `sort | uniq -c | sort -rn | head -n 5`. Repère les « faux doublons » (FORCES, espaces) : c'est la preuve qu'il faut **nettoyer** les données.
8. Liste tous les fichiers `.py` du dossier `WEEK1` avec `find`, et enregistre la liste dans `bash_lab/mes_scripts.txt`.
9. Écris `bash_lab/rapport.sh` : il prend un nom de fichier en argument (`$1`) et affiche son nombre de lignes et ses 3 premières lignes.
10. Rends-le exécutable et lance-le sur `data/raw/resultats.txt`.

11. Mini-test corrigé

1. Tu es dans `~/AI-ENGINEER-2027/WEEK1/DAY4`. Où t'amène `cd ../..` ? ★

Dans `~/AI-ENGINEER-2027` : deux étages au-dessus (DAY4 → WEEK1 → AI-ENGINEER-2027).

2. Quelle différence entre `find` et `grep` ? ★

`find` cherche des fichiers par leur **nom**. `grep` cherche un **texte à l'intérieur** des fichiers.

3. Que fait `ls > liste.txt` si `liste.txt` existe déjà ? ★★

Il l'**écrase**. Pour ajouter à la fin, il faut `>>`.

4. Pourquoi `nom = Joseph` plante dans un script Bash ? ★★

À cause des espaces autour du `=` : Bash croit que `nom` est une commande. Il faut écrire `nom=Joseph`.

5. Que fait `grep -c "ondes" resultats.txt` ? ★★

Il **compte** les lignes qui contiennent exactement « ondes », en respectant les minuscules.

12. À savoir par cœur

1. `pwd` = où suis-je. `ls` = qu'y a-t-il ici. `cd` = je me déplace.
2. `~` maison, `.` ici, `..` l'étage du dessus. `Tab` complète, `↑` rappelle.
3. Chemin absolu = adresse complète. Chemin relatif = « à partir d'ici ».
4. `mkdir -p`, `touch`, `cp -r`, `mv` (déplace ou renomme), `rm` (sans corbeille !).
5. `cat`, `head`, `tail`, `wc -l` pour lire et compter.
6. `find` trouve le livre, `grep` trouve la phrase.
7. `|` enchaîne les commandes. `>` écrase un fichier, `>>` ajoute.
8. Script : `#!/bin/bash`, `$1`, `chmod +x`, `./script.sh`. Pas d'espace autour de `=`.
9. `$PATH` = la liste des dossiers où le terminal cherche les programmes.
